

HERRAMIENTA COMPUTACIONAL PARA SIMULACIÓN DE PROBLEMAS MULTIFÍSICOS NO LINEALES BASADA EN REDES DE FLUIDOS

Pablo Pacheco Pérez, Marcelo Gallardo Maluenda y Cristian
Chávez Toro

XXIV Congreso Chileno de Mecánica Computacional

Universidad de La Serena

September 22, 2026

Contenido

Introducción

- Importancia

- Objetivos

Código NetSystems

- Estructura: Sistemas Lineales

- Estructura: Sistemas no Lineales

- Estructura: Sistemas Acoplados

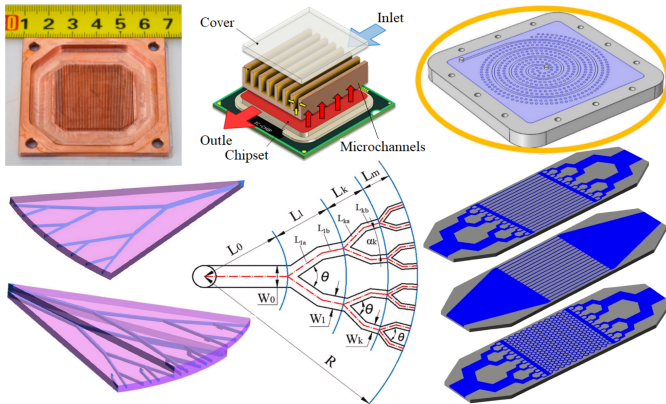
Ejemplos

- Sistemas Lineales

- Sistemas no Lineales

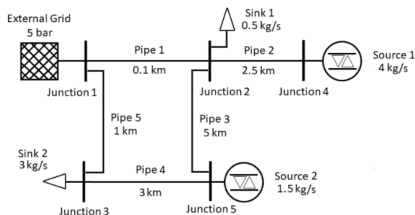
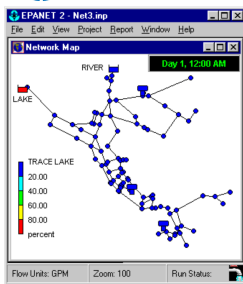
- Sistemas no Lineales

Conclusiones



1. Rostami, S., et al. (2022). Effect of using a heatsink with nanofluid flow and PCM on thermal management of plate lithium-ion battery. *Journal of Energy Storage*, 55, 105650.
2. Akhtari, H., & Ghazani, A. S. (2025). Numerical investigation of cooling enhancement in double-layer fractal-like microchannel disk heat sinks... *Results in Engineering*, 25, 103512.
3. Ji, X., et al. (2022). Experimental study of ultralow flow resistance fractal microchannel heat sinks for electronics cooling. *Int. J. Thermal Sciences*, 181, 107775.
4. Wang, X., et al. (2026). Thermal-hydraulic performance and design screening of a Y-shaped bifurcated honeycomb microchannel heat sink. *Int. Comm. Heat Mass Transfer*, 172.

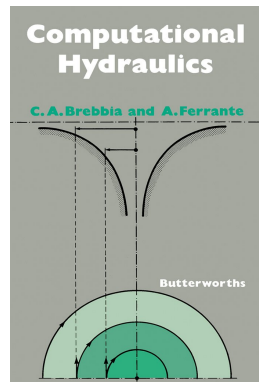
Códigos Disponibles



1. **EPANET**: Rossman, L. A. (2000). *EPANET 2: users manual*. U.S. Environmental Protection Agency, Cincinnati.
2. **pandapipes**: Lohmeier, L., et al. (2020). *pandapipes: An Open-Source Piping Network Calculation Tool*. IEEE Access.

Código NetSystems

- ▶ Código en lenguaje Python
- ▶ Modelos multifísicos acoplados
- ▶ Sistemas lineales y no lineales



Brebbia, C. A., & Ferrante, A. J. (2013). *Computational hydraulics*. Butterworth-Heinemann.

Ecosistema Computacional y Librerías Empleadas

- ▶ **NumPy** Manejo eficiente de arrays n-dimensionales y álgebra lineal base.

Autor Orig.: Travis Oliphant et al.

- ▶ **SciPy** Colección de algoritmos numéricos.

Ref: Virtanen, Gommers et al. (Nature Methods, 2020)

- ▶ **CoolProp** Propiedades termofísicas de fluidos.

Ref: Ian Bell, J. Wronski, S. Quoilin (2014)

- ▶ **Matplotlib** Generación de gráficos 2D/3D.

Creador: John D. Hunter (Computing in Sci. & Eng., 2007)

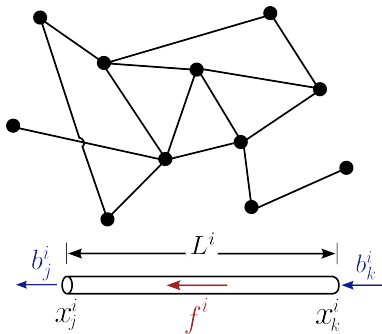


Código NetSystems

Relación fundamental del elemento:

$$\begin{bmatrix} b_k^i \\ b_j^i \end{bmatrix} = k^i \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x_k^i \\ x_j^i \end{bmatrix} \quad (1)$$

- ▶ x_k^i y x_j^i incógnitas del elemento i .
- ▶ b_k^i y b_j^i flujos nodales o términos fuente.
- ▶ f^i flujo a través del elemento i .
- ▶ L^i largo del elemento i .
- ▶ k^i rigidez/conductividad del elemento.



Solución del Sistema Lineal: Partición y Solvers

El sistema global se particiona según las condiciones de contorno:

$$\mathbf{Ax} = \mathbf{b} \rightarrow \begin{bmatrix} \mathbf{A}_E & \mathbf{A}_{EF} \\ \mathbf{A}_{EF}^T & \mathbf{A}_F \end{bmatrix} \begin{bmatrix} \mathbf{x}_E \\ \mathbf{x}_F \end{bmatrix} = \begin{bmatrix} \mathbf{b}_E \\ \mathbf{b}_F \end{bmatrix}$$

- ▶ $\mathbf{x}_F, \mathbf{b}_F$: Incógnitas y fuentes conocidas (Neumann).
- ▶ $\mathbf{x}_E, \mathbf{b}_E$: Nodos Dirichlet y flujos a calcular.
- ▶ $\mathbf{A}_F, \mathbf{A}_E, \mathbf{A}_{EF}$: Submatrices.

Sistema reducido a resolver para $\mathbf{x}_F$:

$$\mathbf{A}_F \mathbf{x}_F = \mathbf{b}_F - \mathbf{A}_{EF}^T \mathbf{x}_E$$

Recuperación de flujos residuales:

$$\mathbf{b}_E = \mathbf{A}_E \mathbf{x}_E + \mathbf{A}_{EF} \mathbf{x}_F$$

NumPy / SciPy Sparse Linear Algebra (`scipy.sparse.linalg`)

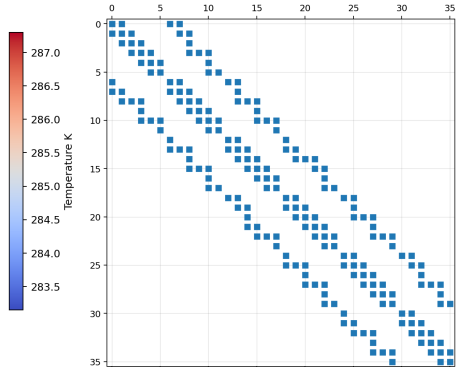
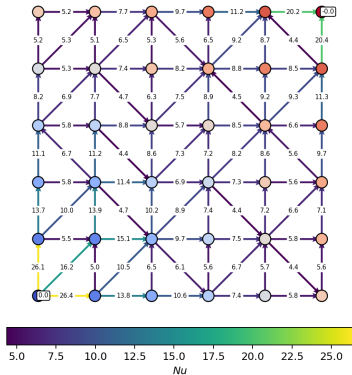
Implementación Computacional

- ▶ **Álgebra Matricial:** *array slicing* con NumPy.
- ▶ **Solvers Lineales (SciPy):**
 - ▶ Directo
 - ▶ Iterativos: `cg`, `bicg`, `bicgstab`, `cgs`, `gmres`, `minres`, `lgmres`, `qmr`, `gcrotmk`, `tfqmr`

Alto Rendimiento

SciPy envuelve implementaciones altamente optimizadas escritas en lenguajes de bajo nivel como Fortran, C y C++. Disfruta de la flexibilidad de Python con la velocidad del código compilado.

► Formato **CSR** (Compressed Sparse Row).



Solución del Sistema No Lineal y Optimización

Partición del sistema global:

$$\mathbf{A}(\mathbf{x})\mathbf{x} = \mathbf{b} \rightarrow \begin{bmatrix} \mathbf{A}_E & \mathbf{A}_{EF} \\ \mathbf{A}_{EF}^T & \mathbf{A}_F \end{bmatrix} \begin{bmatrix} \mathbf{x}_E \\ \mathbf{x}_F \end{bmatrix} - \begin{bmatrix} \mathbf{b}_E \\ \mathbf{b}_F \end{bmatrix} = \mathbf{0}$$

► **Cálculo del Residuo:**

$$\mathbf{R}(\mathbf{x}_F) = \mathbf{A}_F \mathbf{x}_F + \mathbf{A}_{EF}^T \mathbf{x}_E - \mathbf{b}_F = \mathbf{0}$$

- Tras converger $\mathbf{x}_F$, el resto de variables ($\mathbf{b}_E$) se calcula por el método de particiones.

Solvers No Lineales

► **Métodos del Código:**

- `least_squares` (Método TRF)
- Newton-Raphson (Jacobiano numérico)
- Newton-Krylov (`newton_krylov`)

► **Minimización (SciPy):**

Algoritmos de

`scipy.optimize.minimize`

aplicados a la norma $\|\mathbf{R}(\mathbf{x}_F)\|_2$:

- BFGS, CG, Nelder-Mead, Powell, Newton-CG
- L-BFGS-B, TNC
- SLSQP, trust-constr, COBYLA

Referencias: SciPy Optimization and Root Finding (`scipy.optimize.minimize / least_squares`).

Sistemas Advectivos y Acoplamiento Secuencial

Partición del sistema lineal advectivo:

$$\mathbf{A}_{adv} \phi = \mathbf{b}_{adv} \rightarrow \begin{bmatrix} \mathbf{A}_{uu} & \mathbf{A}_{uk} \\ \mathbf{A}_{ku} & \mathbf{A}_{kk} \end{bmatrix} \begin{bmatrix} \phi_u \\ \phi_k \end{bmatrix} = \begin{bmatrix} \mathbf{b}_u \\ \mathbf{b}_k \end{bmatrix}$$

Balance advectivo discreto por nodo:

$$\sum_{e \in \mathcal{E}_i} |Q_e| \gamma_e (\phi_i - \phi_{up}) = S_i$$

- ▶ Q_e : Caudal volumétrico calculado previamente.
- ▶ γ_e : Propiedad advectiva (v.g., ρc_p en transporte térmico).

Solución y Convergencia

▶ Solvers Lineales (SciPy)

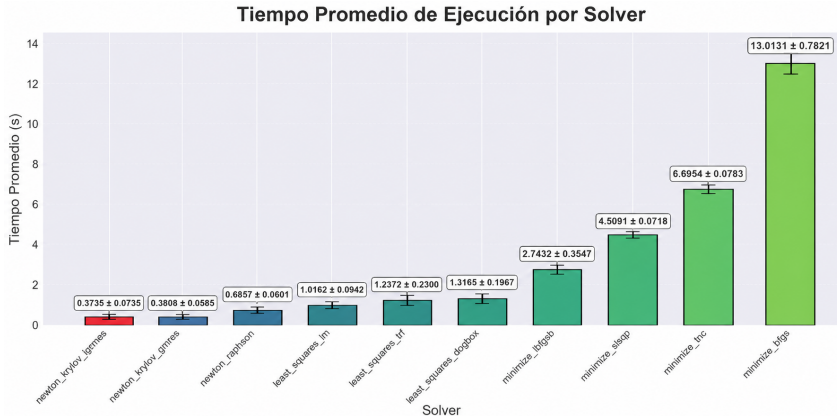
- ▶ Directo: `spsolve`
- ▶ Iterativos Krylov: `cg`, `gmres`, `bicgstab`

▶ Lazo Segregado Estacionario:

1. Resolver sistema no lineal $\rightarrow$ Obtener $\mathbf{Q}$.
2. Construir $\mathbf{A}_{adv}$.
3. Resolver sistema advectivo lineal $\rightarrow$ Obtener ϕ^{k+1} .
4. Evaluar parada:

$$\|\phi^{k+1} - \phi^k\|_2 < \varepsilon$$

Estructura: Sistemas Acoplados



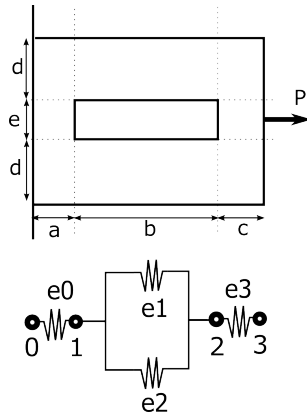
Ejemplo: Estructura 1D Lineal

```

0  import numpy as np
1  import netsystems as ns
2
3  net = ns.Network()
4  connectivity = np.array([[1, 2],
5  [1, 2], [2, 3]])
6  net.set_connectivity(connectivity)
7
8  st = ns.System("structure",
9  system_type="diffusive")
10 st.x_name = "nodal_displacement"
11 st.b_name = "nodal_force"
12 st.element_variable_name = "axial_force"
13
14 st.known_x_nodes = np.array([0])
15 st.known_x_values = np.array([0.0])
16 st.known_b_nodes = np.array([3])
17 st.known_b_values = np.array([800])
18 ...
19 st.element_conductance = A * E / L
20 net.add_system(st)
21 ns.solve_linear_system(net, "structure")
22
23 # Resultados obtenidos:
24 # Disp: [0.0, 8.82e-5, 5.29e-4, 7.06e-4]
25 # Force: [-800, 0.0, 0.0, 800.0]

```

$$F^i = k^i \Delta u \quad \text{donde} \quad k^i = \frac{A^i E}{L^i}$$



Ejemplo: Circuito Eléctrico Resistivo

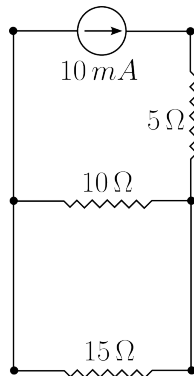
```

0  import numpy as np
1  import netsystems as ns
2  net = ns.Network()
3  connectivity = np.array([[1, 2], [2, 0],
4                           [2, 0]])
5  net.set_connectivity(connectivity)
6  el = ns.System("electric",
7                system_type="diffusive")
8  el.x_name = "voltage"
9  el.b_name = "current"
10 el.element_variable_name = "
    branch_current"
11 el.known_x_nodes = np.array([0])
12 el.known_x_values = np.array([0.0])
13 el.known_b_nodes = np.array([1])
14 el.known_b_values = np.array([0.01])
15 R = np.array([5.0, 10.0, 15.0])
16 el.element_conductance = 1.0 / R
17
18 net.add_system(el)
19 ns.solve_linear_system(net, "electric")
20 # Resultados obtenidos:
21 # Volt: [0.0, 0.11, 0.06]
22 # Branch Curr: [0.01, 0.006, 0.004]

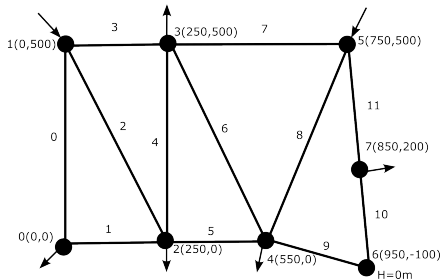
```

Analogía conductiva elemental:

$$\begin{bmatrix} I_i \\ I_j \end{bmatrix} = \frac{1}{R} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} V_i \\ V_j \end{bmatrix} \quad k = \frac{1}{R}$$



Ejemplo: Mecánica de Fluidos - No Lineal



$$V = 0.3545 c D^{0.63} \frac{h^{0.54}}{L^{0.54}} \quad Q = \left(0.2784 \frac{c D^{2.63}}{L^{0.54}} \right) \Delta H^{0.54}$$

$$Q = k \Delta H^{0.54} \quad \text{donde} \quad k = 0.2784 \frac{c D^{2.63}}{L^{0.54}}$$

La ecuación se adapta a la **forma matricial difusiva estándar**, la expresión no lineal se reorganiza como:

$$Q^i = \frac{k^i}{(\Delta H^i)^{0.46}} \Delta H^i$$

Definiendo una **conductividad no lineal efectiva**:

$$\bar{k}^i = \frac{k^i}{(\Delta H^i)^{0.46}}$$

De este modo, la matriz elemental resulta:

$$\begin{bmatrix} Q_k^i \\ Q_j^i \end{bmatrix} = \bar{k}^i \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} H_k^i \\ H_j^i \end{bmatrix}$$

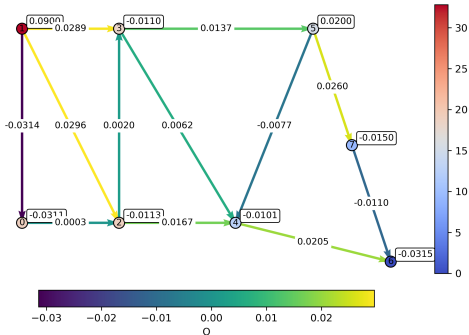
Esta formulación permite resolver el sistema no lineal de forma iterativa actualizando $\bar{k}^i$ en cada iteración.

Ref: Brebbia, C. A., & Ferrante, A. J. (2013). *Computational Hydraulics*. Butterworth-Heinemann. Ejemplo 4.27.

Sistemas no Lineales

$$Q^i = \frac{k^i}{(\Delta H^i)^{0.46}} \Delta H^i \quad \bar{k}^i = \frac{k^i}{(\Delta H^i)^{0.46}}$$

$$\begin{bmatrix} Q_k^i \\ Q_j^i \end{bmatrix} = \bar{k}^i \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} H_k^i \\ H_j^i \end{bmatrix}$$



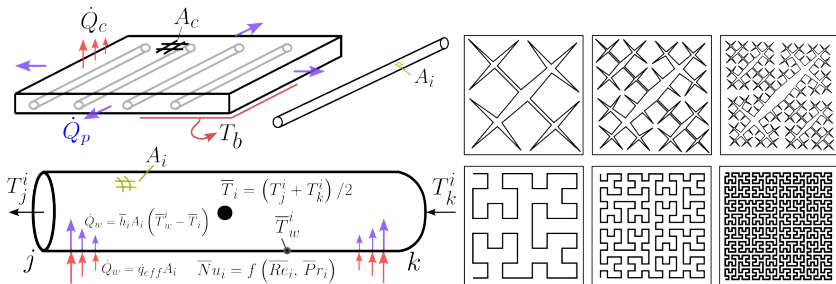
```

0  fluid = ns.System("fluid",
1      system_type="diffusive")
2  fluid.x_name = "H"
3  fluid.b_name = "C"
4  fluid.element_variable_name = "Q"
5
6  def fluid_conductance(net):
7      H = net.get("H")
8      # Indices de conectividad
9      i = net.connectivity[:, 0].
10         astype(int)
11      j = net.connectivity[:, 1].
12         astype(int)
13
14      DH = H[i] - H[j]
15      D = net.element_diameter
16      L = net.element_lengths
17      C = 110
18
19      k = (0.2784 * C * D**2.63 / L
20          **0.54 /
21          (np.abs(DH)**0.46 + 1e
22             -12))
23
24      return k
25
26 fluid.
27     element_conductance_function
28     = fluid_conductance
29 net.add_system(fluid)

```

Ejemplo: Mecánica de Fluidos - Transferencia de Calor

Redes de Micro Canales - Enfriamiento



- ▶ $\dot{Q}_c$, $\dot{Q}_p$ calor impuesto en bordes superior y lateral.
- ▶ T_b temperatura impuesta en la base.
- ▶ A_c área de la celda.

- ▶ Caudal de entrada conocido (ajustado según pérdida de carga).
- ▶ Presión de salida $H_{out} = 0$.
- ▶ Fluido de enfriamiento: agua líquida

Modelo hidráulico no lineal

$$Q \propto (\Delta H)^n, \quad n = 0.54$$

$$Q^i = \frac{k^i}{|\Delta H^i|^{0.46}} \Delta H^i = \bar{k}^i \Delta H^i$$

$$\bar{k}^i = \frac{0.2784 C_{HW} D^{2.63}}{L^{0.54} |\Delta H^i|^{0.46}}$$

$$C_{HW} = \left(\frac{5.335 g \pi^2 |Q|^{-0.15} D^{0.13}}{f} \right)^{1/1.85}$$

La matriz elemental mantiene la forma:

$$\begin{bmatrix} Q_k \\ Q_j \end{bmatrix} = \bar{k} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} H_k \\ H_j \end{bmatrix}$$

La conductividad efectiva $\bar{k}$ se actualiza en cada iteración.

Modelo térmico advectivo

$$F = \gamma Q \phi$$

Para transferencia de calor:

$$F = \dot{H}, \quad \phi = T, \quad \gamma = \rho(T, P) c_p(T, P)$$

por lo tanto,

$$\dot{H} = \rho(T, P) c_p(T, P) Q T$$

La matriz resultante es **no simétrica**, ya que el calor se transporta siguiendo la dirección del flujo.

Acoplamiento fluido-calor

$$P = P_0 + \rho g H$$

Fluido	$\rho(T, P), \mu(T, P)$
Calor	$Q, \rho c_p$
Presión	$P = \rho g H + P_0$

Factor de fricción de Darcy

$$Re \leq 2000 : \quad f = \frac{64}{Re}$$

$$Re \geq 4000 : \quad f = \frac{0.25}{\left[\log_{10} \left(\frac{\varepsilon}{3.7D} + \frac{5.74}{Re^{0.9}} \right) \right]^2}$$

$$2000 < Re < 4000 : \quad f = X_1 + R(X_2 + R(X_3 + X_4))$$

$$Y_3 = -0.86859 \ln \left(\frac{\varepsilon}{3.7D} + \frac{5.74}{4000^{0.9}} \right)$$

$$R = \frac{Re}{2000}, \quad Y_2 = \frac{\varepsilon}{3.7D} + \frac{5.74}{Re^{0.9}},$$

$$FA = Y_3^{-2}, \quad FB = FA \left(2 - \frac{0.00514215}{Y_2 Y_3} \right)$$

$$X_1 = 7FA - FB$$

$$X_2 = 0.128 - 17FA + 2.5FB$$

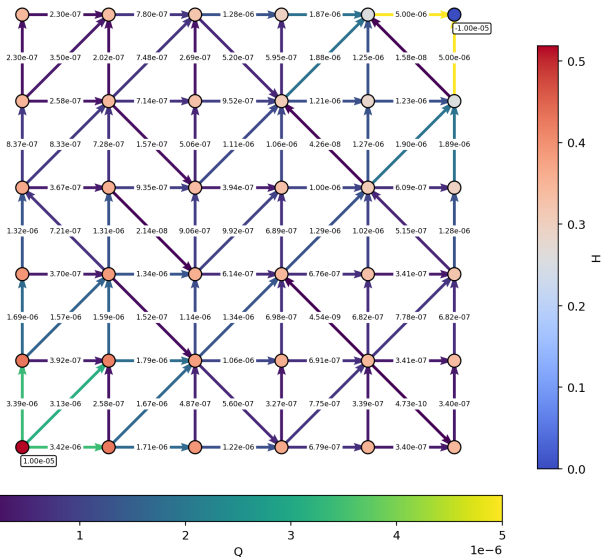
$$X_3 = -0.128 + 13FA - 2FB \quad X_4 = R(0.032 - 3FA + 0.5FB)$$

```

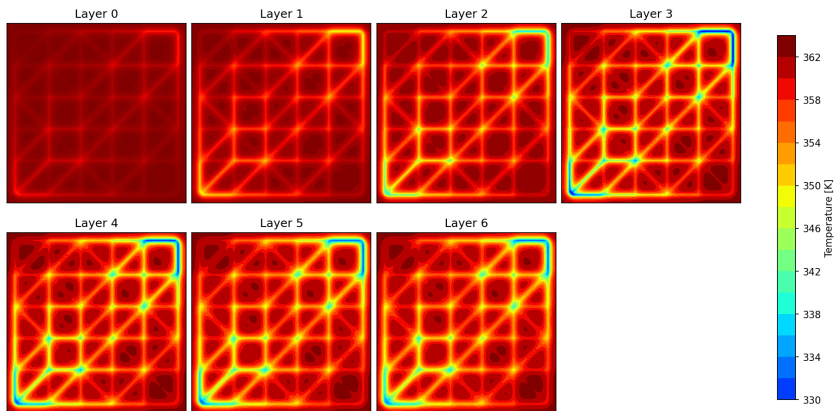
0
1  from CoolProp.CoolProp import
    PropsSI
2
3  def rho_water(net):
4
5      H=net.get("H"); T=net.get("T")
6      g=9.81
7
8      if net.iteration==0:
9          rho=PropsSI("D","T",300,
10                     "P",101325,"water")
11          fluid.rho=np.full(net.n_nodes,
12                             rho)
13      else:
14          P=H*fluid.rho*g+101325
15          fluid.rho=PropsSI("D","T",T,
16                             "P",P,"water")
17      return fluid.rho
18
19  def mu_water(net):
20
21      P=net.get("H")*fluid.rho
22          *9.81+101325
23
24      return PropsSI("V",
25                     "T",net.get("T"),
26                     "P",P,
27                     "water")

```

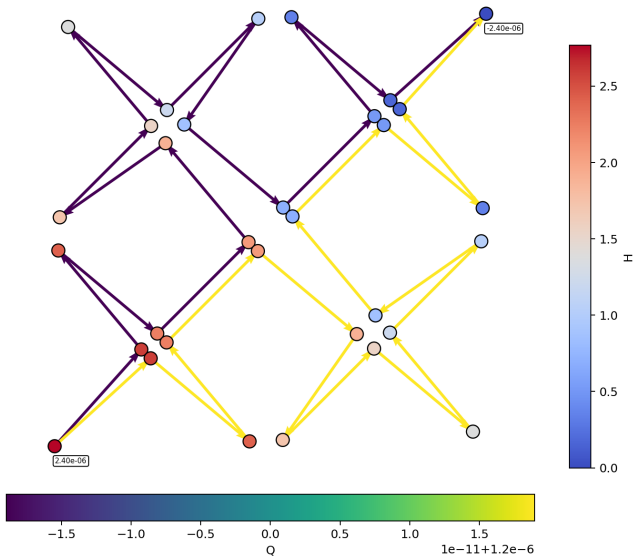
Sistemas no Lineales



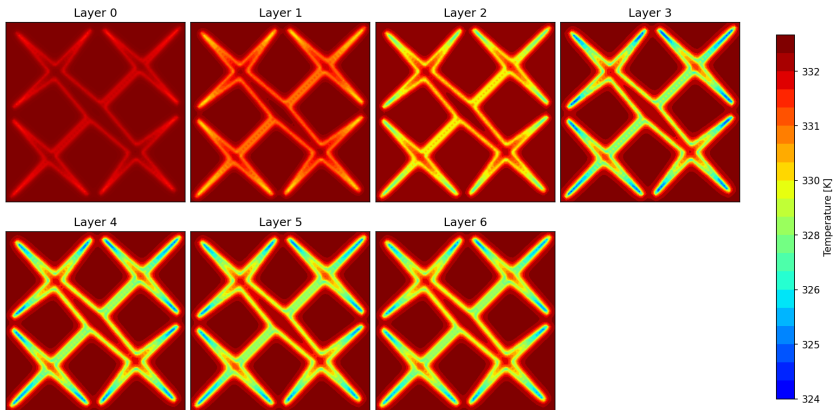
Distribución de Temperatura en Celda de Cobre



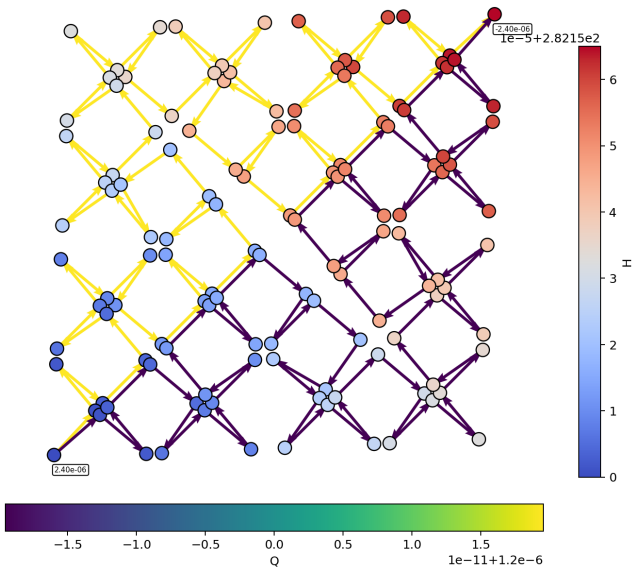
Sistemas no Lineales



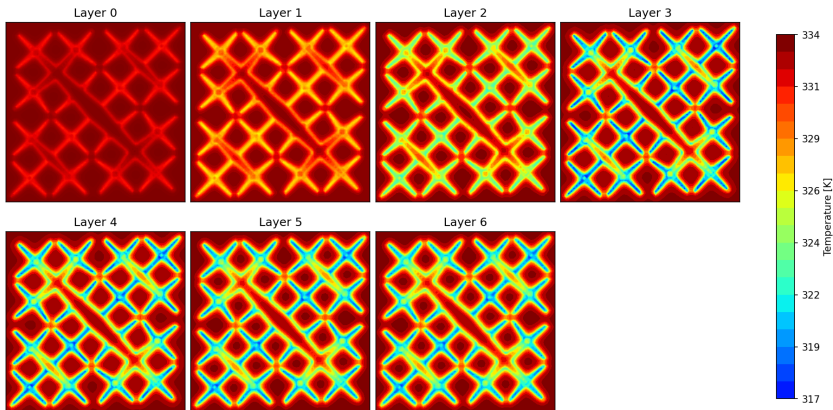
Distribución de Temperatura en Celda de Cobre



Sistemas no Lineales



Distribución de Temperatura en Celda de Cobre



Conclusiones y trabajo futuro

Conclusiones

- ▶ Plataforma computacional abierta, sistemas físicos discretizados mediante redes.
- ▶ Problemas lineales y no lineales mediante ensamblaje matricial y partición automática de ecuaciones.
- ▶ El uso de matrices **sparse (CSR)** y de los solucionadores disponibles en **SciPy**.
- ▶ Incorporar leyes constitutivas, propiedades dependientes del estado y nuevos fenómenos físicos sin modificar el algoritmo principal.

Trabajo futuro

- ▶ Nuevos modelos físicos.
- ▶ Acoplamiento multifísico.
- ▶ Formulaciones transientes.
- ▶ Paralelización y optimización.

HERRAMIENTA COMPUTACIONAL PARA SIMULACIÓN DE PROBLEMAS MULTIFÍSICOS NO LINEALES BASADA EN REDES DE FLUIDOS

Pablo Pacheco Pérez, Marcelo Gallardo Maluenda y Cristian
Chávez Toro

XXIV Congreso Chileno de Mecánica Computacional

Universidad de La Serena

September 22, 2026